```
/*
================================================
============================
   PROJECT 2: HOSPITAL / HEALTHCARE
ANALYTICS (2026)
   Dialect : PostgreSQL 14+
   Skills  : Schema design, Joins, CTEs,
Window Functions (LAG, NTILE,
             RANK, running totals),
readmission & cohort analysis, views
   NOTE    : All data is synthetic. No real
patient information.

================================================
============================ */

-- ---------------------------------------
-----------------------------
-- 0. SCHEMA
-- ---------------------------------------
-----------------------------
DROP TABLE IF EXISTS appointments,
admissions, patients, doctors, departments
CASCADE;

CREATE TABLE departments (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(40) NOT NULL
);
```

```
CREATE TABLE doctors (
    doctor_id   INT PRIMARY KEY,
    doctor_name VARCHAR(50) NOT NULL,
    dept_id     INT REFERENCES
departments(dept_id)
);

CREATE TABLE patients (
    patient_id INT PRIMARY KEY,
    gender     CHAR(1) CHECK (gender IN
('M','F')),
    dob        DATE NOT NULL,
    city       VARCHAR(30),
    insurance  VARCHAR(20)
);

CREATE TABLE admissions (
    admission_id   INT PRIMARY KEY,
    patient_id     INT REFERENCES
patients(patient_id),
    doctor_id      INT REFERENCES
doctors(doctor_id),
    admit_date     DATE NOT NULL,
    discharge_date DATE NOT NULL,
    diagnosis      VARCHAR(40),
    total_bill     NUMERIC(10,2),
    CHECK (discharge_date >= admit_date)
);

CREATE TABLE appointments (
    appt_id    INT PRIMARY KEY,
    patient_id INT REFERENCES
patients(patient_id),
    doctor_id  INT REFERENCES
```

```
doctors(doctor_id),
    appt_date  DATE NOT NULL,
    status     VARCHAR(12) CHECK (status IN
('Completed','Cancelled','No-Show'))
);

-- ----------------------------------------
----------------------------
-- 1. SYNTHETIC DATA
-- ----------------------------------------
----------------------------
INSERT INTO departments VALUES
 (1,'Cardiology'),(2,'Pulmonology'),
(3,'Orthopedics'),
 (4,'Endocrinology'),(5,'Neurology'),
(6,'General Surgery');

INSERT INTO doctors
SELECT g, 'Dr. ' ||
(ARRAY['Rao','Sharma','Khan','Iyer','Patel'
,'Singh',

'Mehta','Das','Nair','Gupta','Joshi','Verma
'])[g],
       1 + (g % 6)
FROM generate_series(1, 12) g;

INSERT INTO patients
SELECT g,
       CASE WHEN g % 2 = 0 THEN 'F' ELSE
'M' END,
       DATE '1940-01-01' + ((g * 211) %
25000),
```

```
(ARRAY['Mumbai','Delhi','Pune','Chennai','K
olkata'])[1 + (g % 5)],

(ARRAY['Private','Government','None'])[1 +
(g % 3)]
FROM generate_series(1, 500) g;

INSERT INTO admissions
SELECT g,
       1 + ((g * 17) % 500),
       1 + ((g * 5) % 12),
       DATE '2026-01-01' + ((g * 11) %
270),
       DATE '2026-01-01' + ((g * 11) % 270)
+ (1 + (g * 5) % 10),

(ARRAY['Diabetes','Hypertension','Pneumonia
','Fracture','Asthma',
              'Cardiac
Arrest','Appendicitis','Migraine'])[1 + (g
% 8)],
       500 + ((g * 733) % 9000)
FROM generate_series(1, 1200) g;

INSERT INTO appointments
SELECT g,
       1 + ((g * 13) % 500),
       1 + ((g * 7) % 12),
       DATE '2026-01-01' + ((g * 3) % 300),
       CASE WHEN g % 10 IN (0)   THEN 'No-
Show'
            WHEN g % 10 IN (1,2) THEN
'Cancelled'
            ELSE 'Completed' END
```

```
FROM generate_series(1, 2000) g;

-- ----------------------------------------
----------------------------
-- 2. HOSPITAL KPIs
-- ----------------------------------------
----------------------------
SELECT
COUNT(*)
     AS total_admissions,
       COUNT(DISTINCT
patient_id)                      AS
unique_patients,
       ROUND(AVG(discharge_date -
admit_date), 2)      AS
avg_length_of_stay_days,
       ROUND(SUM(total_bill),
2)                       AS total_revenue,
       ROUND(AVG(total_bill),
2)                       AS avg_bill
FROM admissions;

-- ----------------------------------------
----------------------------
-- 3. DEPARTMENT & DOCTOR PERFORMANCE
-- ----------------------------------------
----------------------------
-- 3.1 Revenue and patient load by
department
SELECT d.dept_name,

COUNT(*)
AS admissions,
       ROUND(SUM(a.total_bill),
```

```
2)                AS revenue,
       ROUND(AVG(a.discharge_date -
a.admit_date), 1) AS avg_los,
       ROUND(100.0 * SUM(a.total_bill) /
SUM(SUM(a.total_bill)) OVER (), 2) AS
pct_revenue
FROM admissions a
JOIN doctors doc    USING (doctor_id)
JOIN departments d  ON d.dept_id =
doc.dept_id
GROUP BY d.dept_name
ORDER BY revenue DESC;

-- 3.2 Top doctor in every department (RANK
+ PARTITION BY)
WITH doc_stats AS (
    SELECT d.dept_name, doc.doctor_name,
           COUNT(*) AS patients_treated,
           SUM(a.total_bill) AS revenue
    FROM admissions a
    JOIN doctors doc   USING (doctor_id)
    JOIN departments d ON d.dept_id =
doc.dept_id
    GROUP BY d.dept_name, doc.doctor_name
),
ranked AS (
    SELECT *, RANK() OVER (PARTITION BY
dept_name ORDER BY patients_treated DESC)
AS rnk
    FROM doc_stats
)
SELECT dept_name, doctor_name,
patients_treated, ROUND(revenue, 2) AS
revenue
```

```
FROM ranked WHERE rnk = 1
ORDER BY patients_treated DESC;

-- ----------------------------------------
----------------------------
-- 4. DIAGNOSIS & DEMOGRAPHICS
-- ----------------------------------------
----------------------------
-- 4.1 Most common diagnoses with average
cost
SELECT diagnosis,
       COUNT(*) AS cases,
       ROUND(AVG(total_bill), 2) AS
avg_cost,
       ROUND(AVG(discharge_date -
admit_date), 1) AS avg_los
FROM admissions
GROUP BY diagnosis
ORDER BY cases DESC;

-- 4.2 Age-group analysis (age calculated
at admission)
WITH aged AS (
    SELECT a.*, DATE_PART('year',
AGE(a.admit_date, p.dob))::int AS
age_at_admit
    FROM admissions a JOIN patients p USING
(patient_id)
)
SELECT CASE WHEN age_at_admit < 18 THEN '0-
17'
            WHEN age_at_admit < 40 THEN
'18-39'
            WHEN age_at_admit < 60 THEN
```

```
'40-59'
            ELSE '60+' END AS age_group,
       COUNT(*) AS admissions,
       ROUND(AVG(total_bill), 2) AS
avg_bill
FROM aged
GROUP BY 1
ORDER BY MIN(age_at_admit);

-- 4.3 Gender x insurance split
SELECT p.gender, p.insurance, COUNT(*) AS
admissions,
       ROUND(AVG(a.total_bill), 2) AS
avg_bill
FROM admissions a JOIN patients p USING
(patient_id)
GROUP BY ROLLUP (p.gender, p.insurance)
ORDER BY p.gender, p.insurance;

-- ----------------------------------------
----------------------------
-- 5. READMISSION ANALYSIS (headline
portfolio query)
-- ----------------------------------------
----------------------------
-- 5.1 30-day readmissions using LAG
WITH seq AS (
    SELECT admission_id, patient_id,
admit_date, discharge_date, diagnosis,
           LAG(discharge_date) OVER
(PARTITION BY patient_id ORDER BY
admit_date)
               AS prev_discharge
    FROM admissions
```

```
),
flagged AS (
    SELECT *, (admit_date - prev_discharge)
AS days_since_last,
           CASE WHEN admit_date -
prev_discharge BETWEEN 0 AND 30 THEN 1 ELSE
0 END
               AS is_readmit_30d
    FROM seq
)
SELECT COUNT(*) AS total_admissions,
       SUM(is_readmit_30d) AS
readmissions_30d,
       ROUND(100.0 * SUM(is_readmit_30d) /
COUNT(*), 2) AS readmission_rate_pct
FROM flagged;

-- 5.2 Readmission rate by diagnosis
WITH seq AS (
    SELECT diagnosis, admit_date,
           LAG(discharge_date) OVER
(PARTITION BY patient_id ORDER BY
admit_date) AS prev_dc
    FROM admissions
)
SELECT diagnosis, COUNT(*) AS admissions,
       SUM(CASE WHEN admit_date - prev_dc
BETWEEN 0 AND 30 THEN 1 ELSE 0 END) AS
readmits,
       ROUND(100.0 * SUM(CASE WHEN
admit_date - prev_dc BETWEEN 0 AND 30 THEN
1 ELSE 0 END)
             / COUNT(*), 2) AS
readmit_rate_pct
```

```
FROM seq
GROUP BY diagnosis
ORDER BY readmit_rate_pct DESC;

-- 5.3 High-frequency patients (3+
admissions)
SELECT patient_id, COUNT(*) AS visits,
ROUND(SUM(total_bill), 2) AS lifetime_bill
FROM admissions
GROUP BY patient_id
HAVING COUNT(*) >= 3
ORDER BY visits DESC, lifetime_bill DESC
LIMIT 15;

-- ----------------------------------------
----------------------------
-- 6. TRENDS & CAPACITY
-- ----------------------------------------
----------------------------
-- 6.1 Monthly admissions, revenue and MoM
change
WITH m AS (
    SELECT DATE_TRUNC('month',
admit_date)::date AS month,
           COUNT(*) AS admissions,
SUM(total_bill) AS revenue
    FROM admissions GROUP BY 1
)
SELECT month, admissions, ROUND(revenue, 2)
AS revenue,
       ROUND(100.0 * (revenue -
LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER
(ORDER BY month), 0), 2) AS
```

```
revenue_mom_pct,
       ROUND(SUM(revenue) OVER (ORDER BY
month), 2) AS ytd_revenue
FROM m
ORDER BY month;

-- 6.2 Daily bed occupancy (patients in
hospital each day)
WITH days AS (
    SELECT generate_series(DATE '2026-01-
01', DATE '2026-10-31', INTERVAL '1
day')::date AS day
)
SELECT d.day, COUNT(a.admission_id) AS
patients_in_hospital
FROM days d
LEFT JOIN admissions a ON d.day BETWEEN
a.admit_date AND a.discharge_date
GROUP BY d.day
ORDER BY d.day;

-- 6.3 Bill quartiles with NTILE: who are
the high-cost admissions?
WITH q AS (
    SELECT admission_id, diagnosis,
total_bill,
           NTILE(4) OVER (ORDER BY
total_bill) AS bill_quartile
    FROM admissions
)
SELECT bill_quartile, COUNT(*) AS
admissions,
       MIN(total_bill) AS min_bill,
MAX(total_bill) AS max_bill
```

```
FROM q GROUP BY bill_quartile ORDER BY
bill_quartile;

-- ---------------------------------------
----------------------------
-- 7. APPOINTMENTS: NO-SHOW ANALYSIS
-- ---------------------------------------
----------------------------
SELECT d.dept_name,
       COUNT(*) AS appointments,
       ROUND(100.0 * COUNT(*) FILTER (WHERE
ap.status = 'No-Show')   / COUNT(*), 2) AS
no_show_pct,
       ROUND(100.0 * COUNT(*) FILTER (WHERE
ap.status = 'Cancelled') / COUNT(*), 2) AS
cancelled_pct
FROM appointments ap
JOIN doctors doc   USING (doctor_id)
JOIN departments d ON d.dept_id =
doc.dept_id
GROUP BY d.dept_name
ORDER BY no_show_pct DESC;

-- ---------------------------------------
----------------------------
-- 8. REUSABLE VIEW FOR DASHBOARDS
-- ---------------------------------------
----------------------------
CREATE OR REPLACE VIEW vw_admission_summary
AS
SELECT a.admission_id, a.admit_date,
a.discharge_date,
       (a.discharge_date - a.admit_date) AS
length_of_stay,
```

```
       a.diagnosis, a.total_bill,
       p.gender, p.city, p.insurance,
       DATE_PART('year', AGE(a.admit_date,
p.dob))::int AS age_at_admit,
       doc.doctor_name, d.dept_name
FROM admissions a
JOIN patients p    USING (patient_id)
JOIN doctors doc   USING (doctor_id)
JOIN departments d ON d.dept_id =
doc.dept_id;
```